Design Rationale: Bridge + Adapter for Crop-Field Drone Missions

Assignment 3 - individual project. Implementor: FlightController. Adapted class: VendorDroneSdk. Complexity module: dynamic implementor selection.

1. Problem chosen

A farm-technology service flies drones over crop fields. There are several **kinds of mission** (mapping with photos, low-altitude spraying, perimeter patrol) and several **ways to fly them** (an in-memory simulator for dry runs, an exporter that writes a ground-station waypoint plan, and a real drone controlled through a manufacturer SDK). Missions and flight back-ends must vary independently, and the manufacturer SDK cannot be changed.

2. Why neither pattern is enough alone

Bridge alone. Bridge separates *FieldMission* (Abstraction; Mapping, Spraying, PerimeterPatrol as Refined Abstractions) from *FlightController* (Implementor; Simulator, WaypointFile, Vendor). It assumes that every implementor already speaks the Implementor interface. The SDK does not, so without a converter I would either bend the interface to the SDK (breaking the two native back-ends) or let missions call the SDK directly (leaking vendor types into the abstraction).

Adapter alone. An adapter makes one SDK usable, but does nothing about the M x N combinations: 3 missions x 3 back-ends is 9 subclasses (or if/else on the back-end inside each mission), and a fourth back-end costs 3 more classes. With Bridge it is 3 + 3 classes, and each new variant costs one class.

The two solve different problems: Bridge removes the combinatorial explosion, Adapter makes the incompatible class a legal member of the Implementor hierarchy.

3. Why the wrapped implementation is genuinely incompatible

VendorDroneSdk differs from the contract in every dimension required by the assignment, not only in names:

Aspect	FlightController (contract)	VendorDroneSdk (adaptee)
Names	arm, takeOff, goTo, triggerPayload, returnToHome	powerOn, liftOff, navigate, firePayload, goHome
Arguments	goTo(GeoPoint, altitude)	navigate(altitude, lon, lat): reordered, lon before lat
Units / types	altitude in meters (double)	feet; liftOff takes int feet (rounding)
Payload	PayloadCommand enum	numeric channel: 1 shutter, 2 spray open, 3 spray close
Failure	FlightException + FailureReason	SdkStatus with numeric code (0 = ok); may be null; may throw runtime exceptions

VendorDroneAdapter converts units and argument order, maps payload commands to channels, and translates every failure into a *FlightException* (17 to LINK_LOST, 23 to BATTERY_LOW, 31 to OUT_OF_BOUNDS, 40 to PAYLOAD_FAULT, 44/52 to COMMAND_REJECTED, anything else, a null status or an SDK runtime exception to UNKNOWN). Messages contain only the operation and the contract reason, so no vendor code or text leaks. *FieldMission* and its subclasses depend only on *FlightController*; a test scans their sources to keep it that way.

4. Required complexity module: dynamic implementor selection

The input is a text request such as `spray vendor:UN-4471 swLat,swLon neLat,neLon`. The scheme before the colon selects the implementor (*sim*, *file*, *vendor*) and the first word selects the mission. *MissionLauncher* asks two *NamedRegistry* objects for providers and never names a concrete class, so the client never hard-codes a back-end. The adapted implementor is chosen exactly like the native ones.

5. Open/Closed Principle on both axes

New abstraction: *PerimeterPatrolMission* was added as a new subclass and a single register line; no existing class changed. **New implementor:** a new controller is one class plus one register line. Two tests demonstrate this with classes defined only in the test code (a new back-end serving all three

missions, and a new mission running on the simulator and on the adapted vendor drone). The only place that lists shipped classes is the composition root *DefaultWiring*.

6. One limitation

The *FlightController* contract is a lowest-common-denominator, synchronous, command-by-command interface with no telemetry (battery level, position feedback). Missions therefore cannot plan around remaining battery or monitor the flight; they only learn about a problem after a command fails, and then perform a best-effort safe stop (sprayer off, return home). Supporting telemetry would require extending the Implementor interface, which every implementor, including the adapter, would then have to satisfy.